데이터엔지니어 이재웅



이재웅

서울과학기술대학교 산업정보시스템전공 졸업
Microsoft Data School 4기 수료

현장의 문제를 데이터와 기술로 해결했습니다.

데이터의 목적에 맞게 처리 구조를 설계하고, 안정성과 효율을 수치로 확인해 왔습니다.

Tech Stack

01 · Language & Query

Python · SQL

02 · Data Engineering

Spark · Airflow Databricks

03 · Cloud & Infrastructure

Azure · Docker

자격사항 빅데이터분석기사 · SQLD · ADsP | TOEIC 890 · TOEIC Speaking AL

현장의 문제에서 시작한 프로젝트

전력 모니터링과 굴착현장 안전관리, 방역 배차에 필요한 데이터 흐름과 서비스를 구현했습니다.

01 · STREAMING

스마트팩토리 전력 실시간 관제

실시간 데이터부터 의사결정으로 이어지는 파이프라인

Event Hub | Stream Analytics | Docker | Azure SQL DB

1분 데이터 280만 건

전력 사용량 예측 딥러닝 모델 Docker Container화로 비용 절감

02 · EDGE–CLOUD

Gapguard : 스마트 계측 시스템

클라우드가 끊겨도 동작하는 Edge–Cloud 시스템

Azure IoT Hub | Edge Computing | CI/CD | Data Explorer

메시지 수 99% 감소

3-worker 전환 후 Edge 내 테스트 결과

03 · DATA TO SERVICE

가축전염병 위험 예측/배차 최적화

분석 결과를 실제 업무 흐름으로 연결한 서비스

Databricks | PostgreSQL | FastAPI | React / Typescript

효과 14.7%p 증가

위험도 가중 커버리지 개선



스마트팩토리 전력 실시간 모니터링

프로젝트 정의

스마트팩토리 설비에서 발생하는 전력 데이터를 실시간 수집해 활용하는 파이프라인

프로젝트 배경

설비별 센서에서 발생하는 전력 데이터를 실시간 흐름과 활용으로 연결

AS-IS

- 주기적 전력 상태 확인
- 이상 발생 후 원인 확인까지 시간이 소요됨
- 과거 전력 사용량에 기반한 통계적 예측

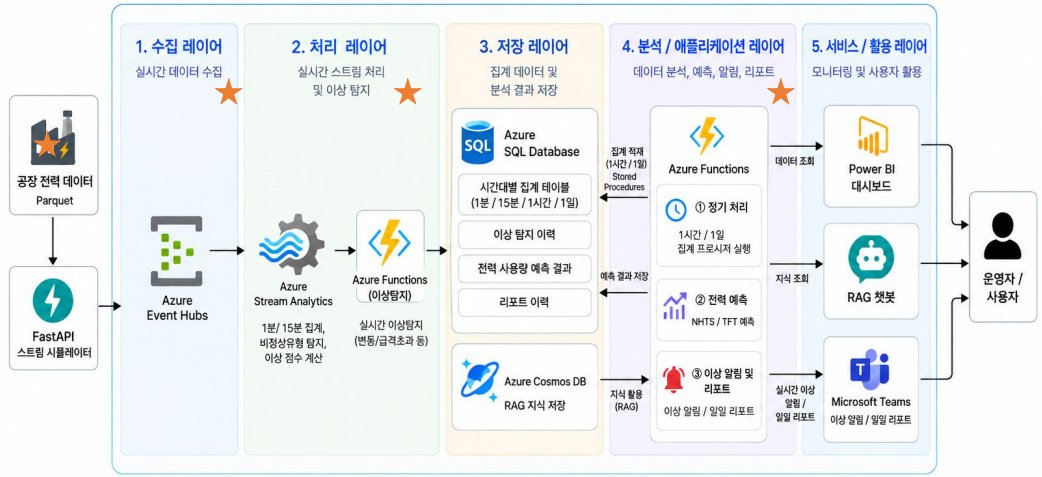
TO-BE

- 실시간 전력 사용 현황 모니터링
- 이상 징후 실시간 탐지 및 원인 확인
- 딥러닝 기반 예측 및 주기적 예측값 갱신

핵심 요구사항

- **전력 사용 현황 실시간 파악:** 설비별 전력 사용량과 피크를 실시간 확인
- **이상 징후 조기 발견:** 전압, 전류, 역률 이상과 급격한 사용량 변화 감지
- **전력 사용량 미리 예측:** 향후 사용량을 예측해 피크 대응과 운영 계획에 활용

프로젝트 아키텍처 ★ 담당 영역



아키텍쳐 구성 중점

- **추론 비용 절감:** Container Apps를 이용한 딥러닝 추론 환경 구성으로 비용 절감
- **트리거 기반 결합 구조:** Timer / SQL Trigger를 이용한 집계 프로시저 실행 / 알림 전송
- **저장소 분리:** 활용 목적과 데이터 형식에 따른 저장소(SQL DB / Cosmos DB) 분리

주요 성과

65% 절감	85% 절감	280만 건 집계
데이터 저장 용량	추론 컴퓨팅 비용	1분 데이터 실시간 처리량
원본 CSV → Parquet 변환 후	Azure ML 대비	수집부터 활용까지

담당 부분 1 : FastAPI 기반 스트림 시뮬레이터

FastAPI 시뮬레이터와 저장 형식 개선

전력 데이터 재생과 이상 시나리오 주입으로 실시간 처리 흐름 테스트

01 시뮬레이터 동작 구조

main.py
스트리밍 시작 / 중지 / 제어
FastAPI 엔드포인트

schemas.py · 요청과 응답 형식 정의
config.py · 환경변수와 공통 설정 관리

데이터 로드
data_loader.py
DuckDB에 저장된 원천 전력 데이터 읽기

이상 주입과 정제
anomaly_injector.py
시나리오 이상 주입 후 결측치 보정 및 정제

스트리밍 제어
stream_manager.py
전송 순서와 속도 제어 5초 단위 데이터 재생

이벤트 전송
eventhub_sender.py
처리한 데이터를 Event Hubs로 전송

Azure Event Hubs

역할별 모듈 분리로 테스트와 제어를 단순화
데이터 로드와 시나리오 주입, 전송 제어를 나누어 구현

02 CSV에서 Parquet으로 저장 형식 전환

원본 전력 데이터의 저장 용량을 줄이기 위해 파일 형식 변경

CSV
텍스트 기반 저장

Parquet
열 단위 저장과 압축

약 65%
데이터 용량 절감

원본 CSV 대비 변환 후 용량 약 35%

변환 과정
원본 CSV를 읽어 Parquet 파일로 저장하고 변환 전후 파일 용량을 비교

담당 부분 2 : 실시간 처리 레이어

파이프라인 핵심

- 5초 전력 스트림을 Stream Analytics에서 실시간 집계
- 활용 목적에 따른 분기(집계 저장, 이상탐지)

실시간 처리 파이프라인

공장 전력 데이터 5초 간격
FastAPI Stream Simulator

Event Hub (power)
전력 스트림 수집 및 전달

Stream Analytics
1분 / 15분 Window 집계
파생 지표 계산
1분 Rule 기반 이상 후보 분리

집계 데이터 경로 1분 2,808,013건 / 15분 187,213건
1m Aggregate
15m Aggregate
Azure SQL
Power BI

이상탐지 경로 이상 후보만 후속 처리
Anomaly Event Hub
Function App
집계 단위별 모델 로드
IQR + Rule Based 점수화
anomaly_prediction
알림 / Power BI

설계 포인트

- **Window Aggregation:** 5초 원천 스트림을 1분 / 15분 데이터셋으로 변환
- **Event-driven Processing:** 이상 후보만 분기해 Function으로 전달
- **저장소 분리:** 집계 데이터와 이상 결과를 목적에 따라 저장

1분 데이터	15분 데이터	CSV → Parquet	Rule Filter 후 점수화
2,808,013건	187,213건	65% 절감	이상 후보만 후속 처리
실시간 집계 데이터	예측 입력용 집계	저장 용량 최적화	

데이터 모델링과 DL 서빙 구조 상세

01 Data Modeling

| 실시간 → 시간 단위 집계 테이블 구성



저장 설계 포인트

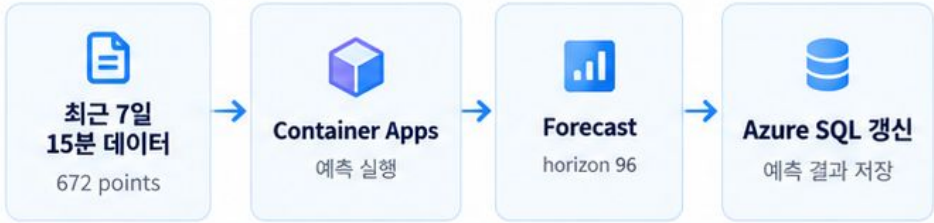
1. 논리명 기준으로 1m / 15m / 1h / 1d Aggregate 분리
2. (equipment_id, window_end_time) 복합키로 장비별 집계 윈도우 식별
3. 중복 적재 방지와 활용 목적별 조회 구조 분리

집계 테이블 요약

그레인(Grain)	생성 주체(Producer)
1m	Stream Analytics
15m	Stream Analytics
1h	Timer Function
1d	Timer Function

02 Scheduled Serving

| 주기적 예측 실행 및 결과 서빙



운영 포인트

1. 7일 15분 데이터 672포인트 조회 후 추론 API 호출
2. 96구간 예측 결과를 Azure SQL에 갱신
3. scale-to-zero 적용으로 비용 약 90% 절감 추산

예측 윈도우 예시

672 points

↓

horizon 96

1m / 15m / 1h / 1d
시간 단위별 집계 구조

(equipment_id, window_end_time)
복합키 기반 중복 방지

약 90%
추론 비용 절감 추산



Edge-Cloud 기반

스마트 계측 시스템

프로젝트 정의

중소규모 굴착현장 안전 구조물의 Edge 기반 실시간 계측 및 Cloud 다현장 관리 시스템

프로젝트 배경

중소규모 굴착현장의 기존 관리 한계 극복, 현장에 맞는 스마트 계측 시스템 개발

AS-IS

- 소규모 굴착현장의 안전관리 공백
- 주 1~2회 수동 계측 및 이상 발생 후 사후 보고
- 네트워크 장애 시 데이터 유실 및 연속성 확보 불가

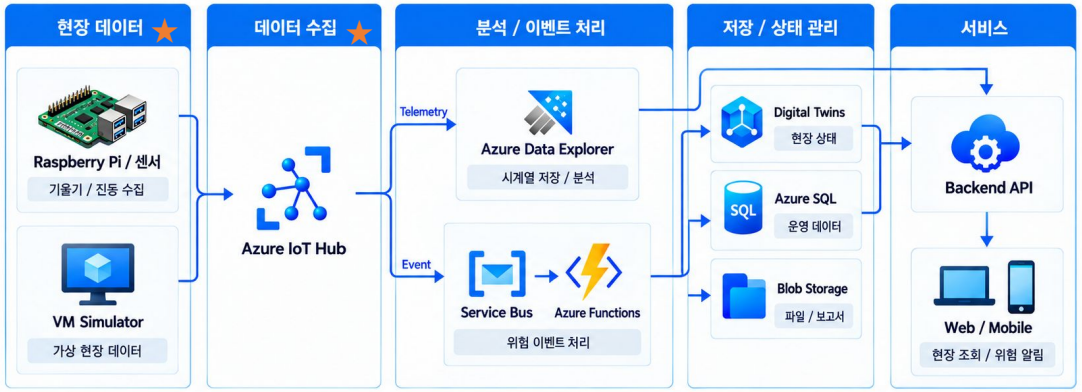
TO-BE

- 현장 Edge 실시간 계측과 위험 판정
- 위험 이벤트 발생 시
- 네트워크 장애 시 데이터 유실 및 연속성 확보 불가

핵심 요구사항

- **실시간 계측 및 현장 판정:** 센서 상시 수집 및 현장에서의 실시간 위험 판정
- **장애 복구와 데이터 무결성:** 네트워크 단절에도 데이터 보존 후 복구
- **다현장 관리:** 다현장 관리를 위한 데이터 통합과 일괄 관리

프로젝트 아키텍처 ★ 담당 영역



아키텍쳐 구성 중점

- **Edge-Cloud 기반 처리:** 센서 데이터 Edge에서 전처리하고 IoT Hub를 통해 클라우드로 전달
- **데이터 특성에 따른 저장소 분리:** 시계열·업무·파일 데이터 저장 주체 분리
- **CI/CD와 IaC 자동화:** Azure Pipelines와 Terraform으로 검증·배포·인프라 관리 자동화

주요 성과

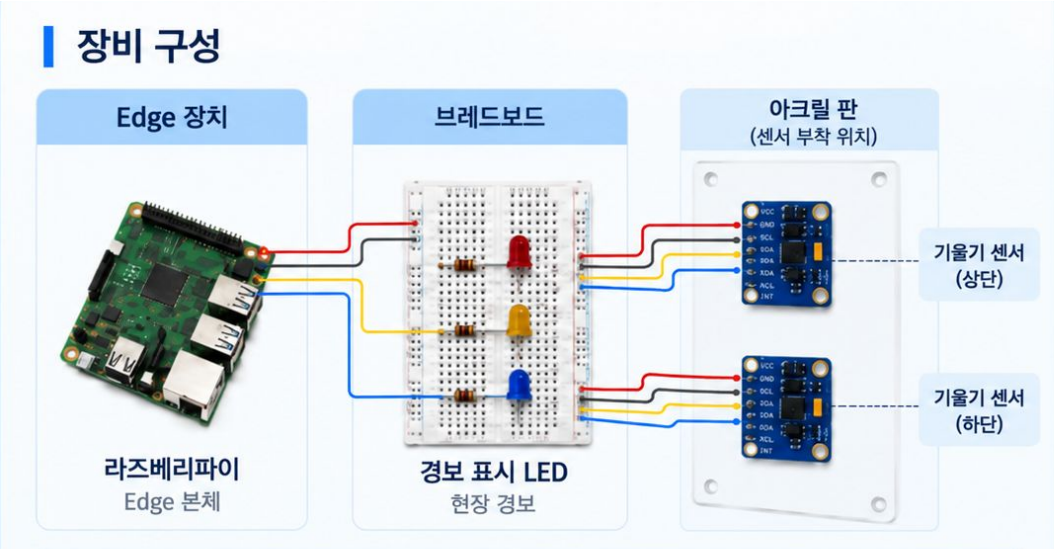
처리 시간 16.2% 개선
3-worker 전환 후 수집-전송 루프

메시지 수집률 13% 개선
3-worker 전환 후 클라우드 적재 결과

10분 테스트 : 100%
네트워크 장애 후 메시지 전량 보존

메시지 양 99%감소
센서의 Raw 데이터 Edge 처리 결과

담당 부분 1 : Edge 도입 배경과 현장 처리



Edge 도입 배경

- Edge의 **실시간 위험 판정**과 **현장 경보 제공**
- 데이터 **무결성** 확보(해시, 연번), 장비 네트워크 단절 시 **데이터 보존** 후 재전송(Outbox)
- 100hz 센서 데이터 **1초 단위 요약**으로 **Cloud 처리량 감소**
  - 메시지 수 **99%** 절감
  - Azure Data Explorer / Azure Digital Twin 업데이트 / Functions 리소스 실행량 99% 감소

전송 과정

100Hz 수집 (MPU6050 x 2(상/하)) → **1초 집계** (진동/기울기 요약) → **현장 판정** (임계값 비교 LED 표시) → **Cloud 전송** (Telemetry, Event)

네트워크 연결 여부와 관계없이 위험 판정과 현장 LED 경보를 유지

담당 부분 2 : 비동기 3-Worker 기반 Edge 처리

문제 직렬 루프에서는 IoT Hub 통신 지연이 센서 수집 간격에 영향을 줌

Collector
100Hz 센서 수집
1초 Window 생성

→

Spooler
진동과 기울기 계산
위험 상태 판정
Outbox 저장

→

Sender
IoT Hub 전송
실패 데이터 재시도
성공 후 삭제

단일 Python 프로세스에서 세 비동기 Task를 독립 실행

3 worker 분리 후 개선

처리 시간 16.2% 감소
루프당 처리 1201 ms → **1007** ms

메시지 수집률 13% 개선
1분 윈도우 sample count 54 → **60**

통신 지연 중에도 수집 주기와 현장 판정 유지

담당 부분 3 : Edge 네트워크 장애 복구

전송 전, 엣지 로컬 SQLite Outbox에 기록해 네트워크 단절과 프로세스 재시작에 대비

먼저 저장
측정값 + 연번, 측정 시점, 해쉬값

→ **단절시**
전송 실패 Outbox 보존

→ **복구**
FIFO 순서로 미전송 재전송

→ **완료**
전송 성공 후 해당 행 삭제

네트워크 단절 테스트

10분	600건	0건
연결 단절	미전송 데이터 보존	전송 실패

검증 결과

FIFO 순서로 Outbox에 보관된 600건을 재전송, 전량 전송 후 Outbox 삭제와 Cloud 적재 확인
프로세스 재시작 후에도 미전송 데이터를 다시 읽어 ADX와 SQL 적재를 확인

현장 판정은 연결 상태와 관계없이 계속 수행

담당 부분 4 : C2D 기반 원격 운영

Cloud의 운영 설정을 Edge에서 검증한 뒤 적용하고 현재 상태를 다시 보고

Web UI
운영 명령

→ **Backend**
요청 전달

→ **IoT Hub**
C2D 라우팅

→ **Edge Listener**
검증과 반영

Direct Method
- 자가점검
- 100Hz 원시 파형 스냅샷

“즉시 실행”

Device Twin
- 임계값 변경
- 전송 주기 전환

“지속 설정 동기화”

IoT 허브 Device Twin 명령 구조

desired → **Edge 검증** → **설정 적용** → **reported**

잘못된 설정은 거부하고 기존 값을 유지
WARNING과 DANGER 상태에서는 Edge의 현장 판정을 계속 수행

담당 범위 Edge Listener, 설정 검증, 적용 결과 reported 반영



가축전염병 위험도

예측 및 배차 최적화

프로젝트 정의

데이터 기반 가축 전염병 위험도 예측 및 방역 차량 배차 최적화 서비스

프로젝트 배경

과거 이력 기반의 농장 위험도 판단, 경험 기반의 배차 최적화, 적은 방역 인력

AS-IS

- 과거 발생 이력 기반 위험도 판정
- 경험 기반 배차 경로 설정

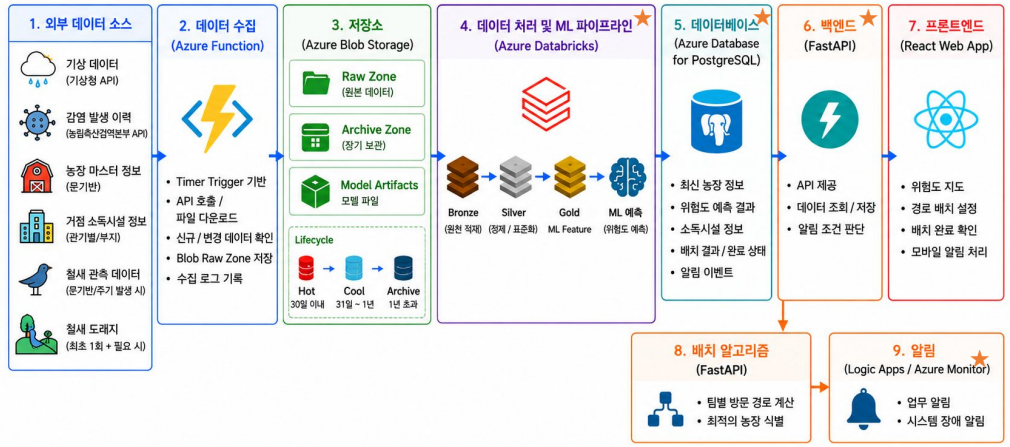
TO-BE

- 데이터(기상, 철새 등) 기반 위험도 예측
- 위험도 판단 근거 제공
- 방역 원칙 제약조건 반영한 배차 최적화

핵심 요구사항

- **위험도 점수 기반 우선순위 판단** : 우선 방문 농가 결정 기준 제공
- **설명 가능한 근거 확보** : 위험 판단에 대한 설명 제공
- **방역 수칙 준수**: 교차오염 방지를 위한 저위험 → 고위험 방문 원칙

프로젝트 아키텍처 ★담당 영역



아키텍처 구성 중점

- **목적 별 저장소 분리:** 원천, 정제, 웹 서빙 목적, 파이프라인의 안정성과 조회 성능 확보
- **Databricks 중심의 ML/배치 워크플로우:** Medallion 아키텍처 기반 정제 피처 관리
- **Azure App Service 기반 웹 CD:** 웹과 백엔드 자동 배포

주요 성과

+14.7%p
위험도 가중 커버리지 개선

+4.8%p
고위험 농장 커버율 개선

-0.4%
총 이동시간 감소

담당 부분 1 : Databricks 데이터 파이프라인 오케스트레이션

파이프라인 오케스트레이션

Job 0(Orchestration) : Job 1~3의 순차 실행 및 파이프라인 전체 모니터링 제어

Job 1

원본 데이터 로드 및 정제

야생조류·농장·발병·기상 원시 데이터 불러오기
결측치 보정·표준화 수행

→

Job 2

피쳐 엔지니어링 및 적재

도메인별 피처 병렬 가공
ML용 최종 통합 마트(Gold) 구축 및 스토리지 아카이빙

→

Job 3(Gold)

ML 추론 및 서빙

통합 피처 기반 일 단위 추론
웹 연동용 서빙 DB 적재

기타 Job : ML 모델 재학습(3개월), 방역 시설 불러오기(6개월)

Databricks ↔ 외부 저장소 연결

Blob Storage

"수집 데이터 원본 csv 파일 저장소"

01 Access Connector 생성
02 IAM 권한 부여(데이터 기여자)
03 Storage Credential 생성
04 External Location 연결

모든 노트북에서 별도 토큰 없이 접근

원본 전달

Databricks

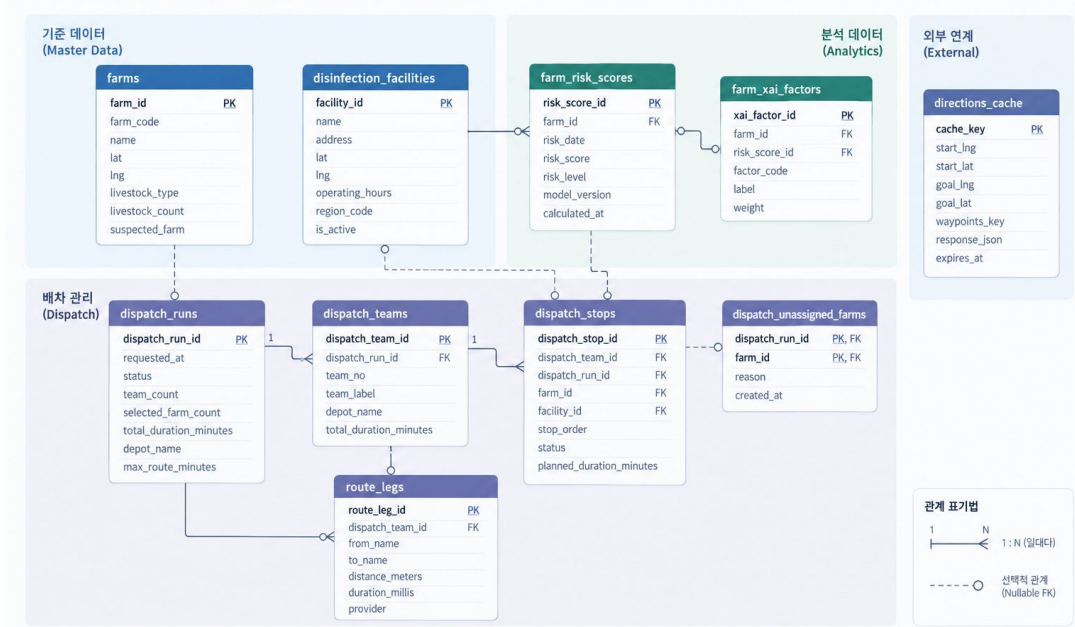
"처리 후 서빙을 안전하게"

01 Secret Scope 생성
02 키 보관(host, port, user, pw)
03 코드상 노출 없는 안전한 연결

ML 결과 서빙

운영용 DB

담당 부분 2 : 서비스용 데이터베이스 설계



주요 테이블 설명

farms 농장 기본 정보와 위치, 축종, 사육 두수, 의심 농장 여부를 관리하는 기준 테이블

farm_risk_scores 농장별 일자 기준 위험도 점수와 위험 등급, 모델 버전을 저장하는 분석 결과 테이블

dispatch_runs 한 번의 방역 배차 실행 정보를 저장하는 테이블로 팀 수, 선택 농장 수, 총 소요시간 등 실행 단위 정보를 관리

dispatch_stops 팀별 방문 농장과 소독시설의 순서, 상태, 예상 소요시간을 저장하는 배차 상세 테이블

route_legs 방문 지점 사이의 거리, 이동시간, 경로 정보를 저장해 실제 이동 경로를 구성하는 테이블

담당 부분 3 : Web (Backend)

주요 REST API

Method	Endpoint	설명
GET	/api/farms	농장 목록과 최신 위험도, XAI 요인 조회
POST	/api/route-assignment	선택 농장과 팀 수 기준 배차 최적화 실행 후 결과 저장
GET	/api/dispatch-runs/{id}	저장된 배차 결과 재조회
POST	/api/directions/multi	확정된 방문 순서를 Naver Directions로 실제 도로 경로 변환

트러블슈팅 : 소요시간 행렬 제작 시 API 호출량 최적화(OSRM 도입과 Naver API 호출량 감축)

문제

🔴배차 최적화를 위해 농장, 출발지, 소독시설 간 다대다 소요시간 행렬 필요
🔴Naver Directions API로 모든 지점 쌍 조회 시 호출량 과다
🔴반복 배차 실행 시 API 호출 제한과 비용 부담 발생

해결

🔵소요시간 행렬 계산을 OSRM Table API로 전환
🔵배차 최적화 끝난 최종 방문 경로만 Naver API 호출
🔵계산용 경로와 시각화용 경로 분리

효과

🟢다수 지점 간 이동시간 조회를 한 번의 Matrix 요청으로 처리
🟢Naver API 호출 축소

담당 부분 4 : 파이프라인 실패와 신규 확진 알림

알림 대상을 서비스 제공자와 이용자로 나누고 서로 다른 경로로 전달

1. 서비스 제공자용 운영 알림

Databricks Job → Webhook → Logic Apps → Teams

1) Databricks Job 알림 : 개별 Job 이름, 작업 ID, 실행 ID 메시지로 전달

Function App → App Insights → Alert Rules → Logic Apps → Teams

2) 수집 함수 실패 알림 : 함수명과 HTTP 응답코드를 파싱해 메시지로 전달

2. 사용자(Web)용 신규 확진 알림

수집 함수 (신규 확진 비교) → 로그 Blob 저장 (로그용 디렉토리) → Blob Trigger (Backend 서빙) → Backend / Web (비상모드 연계)

수집 결과와 기존 발생 이력을 비교해 새 확진만 Backend로 전달

오픈소스 기반 데이터 플랫폼 재구축

As-is
Databricks 중심
Managed Pipeline

→

To-be
Airflow + PySpark + R2 + Delta Lake
오픈소스 기반으로 재구성

Airflow
DAG / Schedule / Retry
- 파이프라인 실행 순서 제어
- 오류 시 재시도 및 로그 관리

↓

PySpark
Transform / Validation
- 데이터 정제 및 변환
- Delta read / write

↓

Cloudflare R2
S3A 기반 Object Storage
Delta Lake 저장소
- S3A Connector 사용
- R2에 Delta Table 저장

↓

Delta Lake Medallion Architecture

- **Bronze** — 원천 데이터 적재 (Raw)
- → **Silver** — 정제 및 표준화 (Cleaned)
- → **Gold** — 분석/서빙용 데이터 (Serving)

01 문제 상황

Parquet은 저장되지만 Delta Table 저장 시 오류 발생

- **Parquet** — read / write 정상
- **Delta Lake** — write 파정에서 오류 발생

↓

02 원인 추적

- **R2 Custom Endpoint** — 엔드포인트 및 경로 설정 확인
- **S3A Connector** — Hadoop AWS 설정 적용
- **의존성 정합성** — Spark / Hadoop / Delta 버전 조정
- **Delta Extension** — Spark 세션에 Delta 확장 적용

↓

03 해결 및 검증

- ✓ **Delta write** 정상 처리
- ✓ **Delta read** 정상 처리
- ✓ **Append** 정상 처리
- ✓ **_delta_log 생성** 정상 생성
- ✓ **Version / History** 정상 조회

구축 결과

- **3 Layer** — Bronze / Silver / Gold Delta Pipeline 구축
- **Delta on R2** — Object Storage 기반 Delta Table 저장 및 운영
- **Airflow** — 일일 DAG / Retry 3회 파이프라인 안정 운영
- **사용 버전** — Spark 3.5.1 / Delta 3.2.0 Hadoop AWS 3.3.4